Chapter 4
Particle Swarm Optimization

Particle Swarm Optimization (PSO) [Kennedy, 2010, Kennedy and Eberhart, 1995] is an optimization algorithm designed for continuous optimization. Like GAs, it is a population-based stochastic method, but unlike GAs it does not take its inspiration from the Theory of Evolution of Darwin, but from the social behavior of bird flocking or fish schooling [Reynolds, 1987]. For instance, one may imagine a flock of birds flying over an area, to find a point to land. In such a situation, defining where the whole swarm should land is a complex problem, since it depends on many pieces of information, such as, for instance, maximizing the availability of food or minimizing the risk of existence of predators. In this context, one can interpret the movement of the birds as a sort of choreography: the birds synchronically move for a period until the best place to land is defined and all the flock lands at once. Given this natural inspiration, PSO is usually not categorized as belonging to the field of Evolutionary Computation, but to the field of Swarm Intelligence.

4.1 The Algorithm

PSO shares many similarities with Evolutionary Computation techniques, such as GAs. For instance, the system is initialized with a population of random solutions and searches for optimal solutions by updating iterations. However, unlike GAs, PSO is not based on the concept of natural selection and does not explore the search space using operators such as crossover and mutation. In PSO, we can imagine that the potential solutions, called particles, "move" through the problem space, mimicking the dynamics of a flock of birds. Several studies have indicated that all birds of a flock searching for a good point to land have a memory of previous points they visited and tend to follow the lead of one of the flock's members. In other words, each member of the flock balances its individual and its swarm knowledge experience, known as social knowledge. While in the natural scenario the quality of a point for the birds to land is quantified by an estimation of the probability of survival, in PSO all solutions have a quality value that is given by the fitness function to be optimized.

L. Vanneschi and S. Silva, *Lectures on Intelligent Systems*, Natural Computing Series,
https://doi.org/10.1007/978-3-031-17922-8_4

Furthermore, PSO uses the concept of velocity, which directs the movement of the particles. Each PSO solution iteratively updates its position, influenced by its "best so far" achieved position and the swarm's current best position. In order to better understand the process, let us first fix some basic concepts and terminology.

- Following the definition of continuous optimization problem given on page 102, individuals of PSO (also called *particles*) are m-dimensional vectors of real numbers.
- A population (also called a *swarm*) is an n-dimensional vector of particles.
- For each $i = 1,2,...,n$ we use the notation $\mathbf{x}_i$ to indicate the *ith* particle of the swarm.
- For each $i = 1,2,...,n$ and for each $j = 1,2,...,m$, we use the notation x_{ij} to indicate the *jth* coordinate of the *ith* particle of the swarm.
- For each $i = 1,2,...,n$ we use the notation $\mathbf{v}_i$ to indicate the velocity of the *ith* particle of the swarm.
- For each $i = 1,2,...,n$ and for each $j = 1,2,...,m$, we use the notation v_{ij} to indicate the *jth* coordinate of the velocity of the *ith* particle of the swarm.
- For each $i = 1,2,...,n$ we use the notation $\mathbf{b}_i$ to indicate the best position (i.e., the position with the best fitness) ever reached by the *ith* particle of the swarm since the beginning of the execution of the algorithm. This position is also called the *local best* of the *ith* particle of the swarm.
- For each $i = 1,2,...,n$ and for each $j = 1,2,...,m$, we use the notation b_{ij} to indicate the *jth* coordinate of the local best of the *ith* particle of the swarm.
- We use the notation $\mathbf{g}$ to indicate the best position ever reached by any particle of the swarm, since the beginning of the execution of the algorithm. This position is also called the *global best*.
- For each $j = 1,2,...,m$, we use the notation g_j to indicate the *jth* coordinate of the best position ever reached by any particle of the swarm.

We have that:

$$\forall i = 1,2,...,n : \mathbf{x}_i \in \mathbb{R}^{\mathrm{m}} \text{ (particle)}, \ \mathbf{v}_{\mathrm{i}} \in \mathbb{R}^{\mathrm{m}} \text{ (velocity)}, \ \mathbf{b}_{\mathrm{i}} \in \mathbb{R}^{\mathrm{m}} \text{ (local best)}$$

Furthermore:

$$\mathbf{g} \in \mathbb{R}^{\mathrm{m}} \text{ (global best)}$$

Finally, the fitness function f returns a real number for each particle, so:

$$f : \mathbb{R}^{\mathrm{m}} \rightarrow \mathbb{R}$$

With this in mind, we are now ready to study the general functioning of PSO. Algorithm 6 presents the pseudocode of PSO for the case of minimization problems. In that algorithm:

- w is a parameter, called the *inertia constant*;

Algorithm 6: The pseudocode for Particle Swarm Optimization, in the case of a minimization problem.

1. $\forall i = 1,2,...,n:$ initialize $\mathbf{x}_i$ and $\mathbf{v}_i$;
2. $\forall i = 1,2,...,n:$ $\mathbf{b}_i := \mathbf{x}_i$;
3. $\mathbf{g} = \text{argmin}_{\mathbf{x}_i} f(\mathbf{x}_i),\ \forall i = 1,2,...,n$;
4. **repeat until** (termination condition)
   - 4.1. **for** each $i = 1,2,...,n$ **do**
     - 4.1.1. $\mathbf{x}_i := \mathbf{x}_i + \mathbf{v}_i$;
     - 4.1.2. $\mathbf{v}_i := w\,\mathbf{v}_i + c_1\,\mathbf{r}_1 \circ (\mathbf{b}_i - \mathbf{x}_i) + c_2\,\mathbf{r}_2 \circ (\mathbf{g} - \mathbf{x}_i)$;
     - 4.1.3. **if** $(f(\mathbf{x}_i) < f(\mathbf{b}_i))$ **then** $\mathbf{b}_i := \mathbf{x}_i$;
     - 4.1.4. **if** $(f(\mathbf{x}_i) < f(\mathbf{g}))$ **then** $\mathbf{g} := \mathbf{x}_i$;
   - **end**
   - **end**
5. **return** $\mathbf{g}$;

- c_1 and c_2 are two parameters, called the *cognitive component* and *social component*, respectively;
- $\mathbf{r}_1$ and $\mathbf{r}_2$ are two m-dimensional vectors of random numbers drawn at every different velocity update event, uniformly from the $[0,1]$ interval;
- the symbol $\circ$ represents the operator of element by element multiplication between two vectors.

Let us now comment on the pseudocode line by line, in order to understand every detail. Lines 1, 2 and 3 initialize the main employed structures. In particular, line 1 initializes the positions and the velocities of all the particles in the swarm. Generally, the particle positions $\mathbf{x}_i$ are initialized randomly. More precisely, for each $i = 1,2,...,n$ and for each $j = 1,2,...,m$, x_{ij} is initialized with a random number drawn with uniform probability from interval $[\alpha_j, \beta_j]$[1]. Also, it is customary to initialize all the velocities at zero ($\mathbf{v}_i := \mathbf{0}$). Line 2 initializes the local best of each particle to the current position (which is also the only position that the particle has occupied so far), while line 3 initializes the global best to the particle with the best fitness in the initial swarm. Lines 4 and 4.1 represent the beginning of the main loops that characterize the algorithm. Line 4 represents the beginning of the external loop, which is repeated until a termination condition is achieved. As is customary in optimization metaheuristics, usually the termination condition is that a global optimum (or a reasonable approximation of it) has been found, or a previously fixed maximum number of iterations has been executed. Line 4.1 represents the beginning of the internal loop, indicating a repetition of the same actions for each particle in the swarm. The actions that are executed are represented by lines 4.1.1 to 4.1.4. Line 4.1.1 modifies the position of each particle, by simply adding the current value

[1] Following the definition of continuous optimization problem given on page 102, usually each coordinate j of the position vectors of the particles has an *a priori* known range of variation $[\alpha_j, \beta_j]$.

of its velocity. This step is useless the first time if the velocity is initialized to zero. Line 4.1.2 modifies the velocity, and it is probably the hardest point to understand in this algorithm. Basically, the new velocity is determined by the sum of three components:

- $w\ \mathbf{v}_i$: this term tends to maintain the movement of the particle in the same direction as in the previous iteration;
- $c_1\ \mathbf{r}_1 \circ (\mathbf{b}_i - \mathbf{x}_i)$: this term tends to let the particle move towards the position of its own local best (cognitive term);
- $c_2\ \mathbf{r}_2 \circ (\mathbf{g} - \mathbf{x}_i)$: this term tends to let the particle move towards the position of the global best of the swarm (social term).

The respective importance of these three terms can be tuned by setting the three parameters w, c_1 and c_2. However, it should be noticed that both the cognitive and the social term have a random component, which can potentially be different for each one of the coordinates, and which bestows stochasticity on the algorithm. Finally, points 4.1.3 and 4.1.4 update the local best and the global best, respectively, if better solutions than the current ones were found. The final step of the algorithm, point 5, returns the current global best as the final result of the search.

The reader should notice that the algorithm is highly parallelizable. In particular, loop 4.1 executes the same action for all the particles independently, and with only one point of synchronization when it comes to updating the global best (point 4.1.4). All the other actions, including the update of the position, the update of the velocity and the calculation of the fitness of the new position (which is typically the most expensive operation, in terms of computational resources) can be run in parallel for each particle, since no synchronization is needed. Furthermore, the parallelism grain can even be finer, i.e., at the level of the single vector coordinates. In fact, the position and velocity update are vectorial operations that can be executed coordinate by coordinate independently. In other words, loop 4.1 could be rewritten as:

for each $i = 1, 2, ..., n$ **do**
    for each $j = 1, 2, ..., m$ **do**
        $x_{ij} := x_{ij} + v_{ij}$;
        $v_{ij} := w\ v_{ij}\ +\ c_1\ r_1\ (b_{ij} - x_{ij})\ +\ c_2\ r_2\ (g_j - x_{ij})$;
    end
    if $(f(\mathbf{x}_i) < f(\mathbf{b}_i))$ **then**
        for each $j = 1, 2, ..., m$ **do** $b_{ij} := x_{ij}$;
    if $(f(\mathbf{x}_i) < f(\mathbf{g}))$ **then**
        for each $j = 1, 2, ..., m$ **do** $g_j := x_{ij}$;
end

where r_1 and r_2 are random numbers (scalars) extracted with uniform distribution from the $[0, 1]$ interval. The possibility of parallelizing PSO both at the particle level and at the coordinate level makes the algorithm potentially very efficient. Given that the calculations are mostly vectorial operations, PSO is particularly suitable for

implementation using Graphic Processing Units (GPUs), which typically makes the algorithm even faster.

As is typical of all the techniques discussed in this book, the pseudocode in Algorithm 6 has to be interpreted just as a general guideline, and several extensions and specifications need to be added when it comes to actually implementing the method. For instance, one has to decide a strategy to "force" the algorithm to respect the limitation of the values of each coordinate position j to the predefined range $[\alpha_j, \beta_j]$. A possibility, whenever Equation 4.1.1 of Algorithm 6 tends to exceed a margin, is to set the value of the coordinate to the margin itself and to invert the sign of the velocity, or reinitialize it to zero, or to a random number. Reinitialization of the position coordinate with a random number in $[\alpha_j, \beta_j]$ is also an option. For instance, if the strategy of setting the coordinate to the margin and inverting the sign of the velocity is adopted, the previous portion of pseudocode could be extended to:

for each $i = 1, 2, ..., n$ **do**
  for each $j = 1, 2, ..., m$ **do**
    $x_{ij} := x_{ij} + v_{ij}$;
    if $x_{ij} < \alpha_j$ **then**
      $x_{ij} := \alpha_j$;
      $v_{ij} := -v_{ij}$;
    else if $x_{ij} > \beta_j$ **then**
      $x_{ij} := \beta_j$;
      $v_{ij} := -v_{ij}$;
    else
      $v_{ij} := w\, v_{ij} \;+\; c_1\, r_1\, (b_{ij} - x_{ij}) \;+\; c_2\, r_2\, (g_j - x_{ij})$;
    end
  end
  if $(f(\mathbf{x}_i) < f(\mathbf{b}_i))$ **then**
    for each $j = 1, 2, ..., m$ **do** $b_{ij} := x_{ij}$;
  if $(f(\mathbf{x}_i) < f(\mathbf{g}))$ **then**
    for each $j = 1, 2, ..., m$ **do** $g_j := x_{ij}$;
end

4.2 Parameter Setting

The choice of PSO parameters can have a large impact on optimization effectiveness. Choosing appropriate parameter values is a complex task that has been the subject of much research. The PSO parameters can be tuned by using another optimization algorithm, or fine-tuned before or during the optimization. Like for any other optimization metaheuristics, appropriate PSO parameter values are highly problem dependent. However, some simple rules of thumb can be discussed. These guidelines

have to be interpreted only as suggestions to practitioners, and can by no means replace parameter tuning. While the dimensions of the particles (m in the previous section) and the range of the different coordinates are usually determined by the problem, concerning the swarm size (i.e., the number of employed particles), it is typical to have significantly smaller values compared to GA populations. A typical range may be 20 to 40 particles, even though for simple problems even smaller values, like for instance 10 particles, can be large enough to get good results, and for some difficult or special problems, one may need larger values, like 100 or 200 particles. The other side of the coin is that in PSO, it is typical to run the algorithm for a larger number of iterations, compared to GAs: from several thousands of iterations to even millions of iterations in the case of parallel and GPU-based implementations.

The learning factors (c_1 and c_2) and inertia weight (w) are probably the hardest parameters to set. A common practice is to start the tuning by using values equal (or "close") to 2 for c_1 and c_2 and close to 1 for w, values that have been shown to be appropriate for several applications. However, other settings were also used in different papers. Even though exceptions can exist, appropriate values for c_1 and c_2 typically range from 0 to 4. Furthermore, it is possible to also introduce another parameter, which is the maximum possible velocity. This can be an appropriate practice in some cases, to avoid too-large "jumps" of the positions of the particles in the search space. When it is used, a typical value for the maximum velocity for each coordinate j is the magnitude of the variation of the position in that dimension. In other words, if the particle position has a variation range in $[\alpha_j, \beta_j]$ for coordinate j, one may set the maximum velocity for that coordinate to $\beta_j - \alpha_j$. A method to force the algorithm to respect this limitation, for instance, can consist in resetting a coordinate of the velocity to zero whenever Equation 4.1.2 in Algorithm 6 tends to exceed the maximum velocity, or to choose a random value between 0 to the maximum velocity.

4.3 Variants

Several variants of the basic PSO algorithm are possible [Cagnoni et al., 2008], and some of them have already been discussed in the previous section. For instance, one may imagine that each method employed to force the algorithm to respect the interval of variation of the position coordinates or to remain within the maximum possible velocity is, in itself, a variant. Besides this, given the clear analogies between PSO and GAs, a lot of effort has been dedicated by researchers to the idea of generating hybrid versions, integrating PSO and GAs into a unique algorithm. If, on the one hand, the update of the position of the particles can be seen as a particular type of mutation, on the other hand it is clear that selection and crossover are completely absent in PSO. The idea of enriching PSO with selection and crossover has been explored, for instance, in [Miranda and Fonseca, 2002, Garg, 2016]. Furthermore, attempts to integrate PSO with other optimization

algorithms have been presented, for instance, in [Krink and Løvbjerg, 2002, Niknam and Amiri, 2010, Zhang and Xie, 2003]. Other research trends consist in using multiple swarms [Vanneschi et al., 2011, Cheung et al., 2014] and multiobjective optimization [Nobile et al., 2012].

Various simplified variants of PSO have been presented, for instance in [Bratton and Blackwell, 2008, Pedersen and Chipperfield, 2010]. Last but not least, PSO has been extended to discrete optimization. A commonly used method is to map the discrete search space to a continuous domain, to apply a classical PSO, and then to demap the result. Such a mapping can be very simple (for example by just using rounded values) or more sophisticated [Roy et al., 2011]. Different approaches for using PSO for discrete optimization were presented, for instance, in [Kennedy and Eberhart, 1997, Clerc, 2004, Jarboui et al., 2008, Chen et al., 2010].

Chapter 8
Genetic Programming

GAs, studied in Chapter 3, are capable of solving many problems and simple enough to allow for solid theoretical studies. Nevertheless, the representation of individuals that characterizes GAs (i.e., the fact that individuals in GAs must be strings of a previously fixed length) can be a limitation for a wide set of applications. In these cases, the most natural representation for a solution is a hierarchical computer program, rather than a string of characters of a fixed length. For example, strings of a static length do not readily support the hierarchical organization of tasks into subtasks typical of computer programs, they do not provide any convenient way of incorporating iteration and recursion, and so on. But above all, GA representation schemes do not have any dynamic variability: the initial selection of string length limits in advance the number of internal states of the system and limits what the system can learn.

This lack of representation power (already recognized in [De Jong, 1988]) is overcome by Genetic Programming (GP) [Koza, 1992]. As with GAs, GP belongs to the field of Evolutionary Computation, but contrarily to GAs, GP operates with general hierarchical computer programs. In other words, GP manages a population of computer programs. These programs are typically initialized randomly, and then evolved, using an algorithm that is inspired by the Theory of Evolution of Charles Darwin, with the objective of automatically discovering programs that are appropriate for solving the problem at hand. So, GP is a method for automatically generating a program that solves a problem, starting from the high-level specifications of the problem itself and, in general, without any intervention from a human programmer. As studied in the previous chapters, data models are particular computer programs, and, as such, GP can be seen as a Machine Learning method. Even though every programming language (e.g., Pascal, Fortran, C, etc.) is capable of expressing and executing general computer programs, Koza chose the *LISP* (*LIS*t *P*rocessing) language to code GP individuals. The reasons for this choice can be summarized as follows:

- Both programs and data have the same form in LISP, so it is possible and convenient to treat a computer program as data in the genetic population.

L. Vanneschi and S. Silva, *Lectures on Intelligent Systems*, Natural Computing Series,
https://doi.org/10.1007/978-3-031-17922-8_8

- This common form of programs and data is a parse tree and this allows us in a simple way to decompose a structure into substructures (subtrees) to be manipulated by the genetic operators.
- LISP facilitates the programming of structures whose size and shape change dynamically and the handling of hierarchical structures.
- Many programming tools were commercially available for LISP.

In other words, GP, as originally defined by Koza, considers individuals as LISP-like tree structures. These structures are perfectly capable of capturing all the fundamental properties of modern programming languages. GP using this representation is called *tree-based GP*[1]. The tree-based representation of genomes, although the oldest and most commonly used, is not the only one that has been employed in GP. In particular, in recent decades, growing attention has been dedicated by researchers to linear genomes [Brameier and Banzhaf, 2001], graph genomes [Miller, 2001], stack-based [Spector and Robinson, 2002] and grammar-based GP [O'Neill and Ryan, 2003].

This chapter is structured as follows: Section 8.1 introduces the main definitions and characteristics of GP and of its operators, including specifications of its typical behaviors and an example of a simple run, described step by step. Section 8.2 presents the most significant theoretical studies that have been performed in GP, known as schema theory. Section 8.3 describes some common typical problems of GP, or GP benchmarks, while Section 8.4 briefly describes some of the current challenges and open issues in GP. Finally, Sections 8.5 and 8.6 present two of the most recent and promising developments of GP.

8.1 The GP Process

In synthesis, the GP paradigm breeds computer programs to solve problems by executing the following steps:

1. Generate an initial population of computer programs (or individuals).
2. Iteratively perform the following steps until the termination criterion has been satisfied:

   2.1. Execute each program in the population and assign it a fitness value according to how well it solves the problem.
   2.2. Create a new population by applying the following operations:
      2.2.1. Probabilistically select a set of computer programs to be reproduced, on the basis of their fitness (selection).
      2.2.2. Copy some of the selected individuals, without modifying them, into the new population (replication[2]).

[1] Of course, modern tree-based GP is implemented in languages like C, C++, Java or Python, rather than LISP.

[2] Originally called reproduction in [Koza, 1992]

2.2.3. Create new computer programs by genetically recombining randomly chosen parts of two selected individuals (crossover).
2.2.4. Create new computer programs by substituting randomly chosen parts of some selected individuals with new randomly generated ones (mutation).
2.2.5. Between the old and new programs, select the ones that will form the new population (survival).

3. The best computer program that appears in any generation is designated as the result of the GP process at that generation. This result may be a solution (or an approximate solution) to the problem.

The attentive reader will not have failed to recognize clear similarities between the general functioning of GAs (presented in Chapter 3) and that of GP. The high-level algorithm is actually extremely similar, with the only major difference typically being that, in GP, the individuals resulting from the application of one genetic operator are not used as input for another genetic operator. In other words, crossover and mutation are not applied sequentially. More specifically, mutation is not applied to the offspring generated by crossover. Instead, some selected individuals undergo crossover, some of them mutation and some of them replication, according to fixed probabilities that are parameters of the algorithm. This is justified by the fact that, as we will study later in this chapter, GP genetic operators are more "disruptive" than those of GAs, and thus applying more than one operator would risk generating individuals that are too different from their parents. Besides this, the different representation of the individuals evolving in the population makes GP deeply different from GAs in its dynamics and interpretation. In the following sections, each part of the GP process is analyzed in detail, specifying the differences and similarities between GP and GAs.

8.1.1 Representation of GP Individuals

In tree-based GP, the set of all the possible structures that can be generated is the set of all the possible trees that can be built recursively from a set of function symbols $\mathscr{F} = \{f_1, f_2, \ldots, f_n\}$ (also called primitive functions, and used to label internal tree nodes) and a set of terminal symbols $\mathscr{T} = \{t_1, t_2, \ldots, t_m\}$ (used to label tree leaves). This potentially infinite search space is usually limited in size, by restricting it to only those trees whose depth is less than or equal to a previously fixed parameter d[3]. Each element in the function set $\mathscr{F}$ takes a fixed number of arguments, specifying its *arity*. Functions may include arithmetic operations ($+$, $-$, $*$, etc.), other mathematical functions (such as `sin`, `cos`, `log`, `exp`), Boolean operations (such as `AND`, `OR`, `NOT`), conditional operations (such as `If-Then-Else`), iterative operations (such as `While-Do`) and/or any other domain-specific functions that may

[3] We recall that the *depth* of a tree is defined as the longest possible path from the root of the tree to one of its leaves.